TECHNICAL PROJECT REPORT / QISKIT FALL FEST 2026

Quantum Portfolio Optimization with Qiskit

QAOA, quantum-kernel machine learning, simulation, and selected IBM Quantum hardware experiments



SUPERPOSITION SQUAD

Rahul Kumar · Alden Antony · Krishna Aggarwal

Submitted in connection with Track 1: Quantum Optimization

IIIT Delhi · 16 October 2026

Report purpose

This report expands the presentation into a technical narrative for faculty review. It documents the problem formulation, classical references, the three experimental levels, simulation and IBM Quantum hardware runs, interpretation of results, limitations, and reproducibility information.

Project repository: github.com/rahulydv39/Qiskit-Fall-Fest-2026

Scope note: all portfolio inputs used in these experiments are synthetic. The project is an educational research prototype, not an investment recommendation. No quantum advantage is claimed.

Executive summary

This project studies three ways to examine portfolio-selection problems with Qiskit: (1) Quantum Approximate Optimization Algorithm (QAOA) on a small portfolio, (2) quantum-kernel machine learning to classify and rank candidate portfolios, and (3) constrained QAOA on a larger synthetic portfolio with sector limits. Each level is paired with a classical reference so that observed outputs can be interpreted quantitatively.

The main methodological principle is to separate implementation from evidence of advantage. We verified candidate solutions against classical search, measured whether sampled bitstrings met portfolio constraints, compared the quantum-kernel classifier with a classical RBF-SVM, and recorded outcomes from selected IBM Quantum jobs. The experiments demonstrate an end-to-end study workflow, but they do not establish quantum advantage.

Project at a glance

Level	Method	Decision problem / dataset	Reference result
Beginner	QAOA	6 stocks; select exactly 3; 20 feasible portfolios	Stocks 2, 3, 6; cost −0.503015
Intermediate	Quantum fidelity kernel + SVM	12 stocks; choose 6; 924 candidate portfolios	Quantum and classical SVM each reported 100% initial split accuracy; classical ranking correlation slightly higher
Advanced	Constrained QAOA	20 stocks; select 10; five sectors with cap 2 each; 7,776 feasible portfolios	Stocks 2, 4, 5, 7, 11, 12, 13, 16, 18, 20; cost −1.111739

Evidence highlights

- Beginner IBM run: 609 of 1,024 shots were valid (59.47%); the exact classical optimum appeared 29 times.
- Intermediate QML: quantum-kernel SVM mean Spearman correlation was 0.9469; classical RBF-SVM was 0.9563 across five recorded ranking evaluations.
- Advanced simulation: the exact classical optimum was sampled once in 4,096 shots; the best sampled cost matched the reference optimum.
- Advanced IBM run: one valid portfolio was observed in 256 shots (0.39%); the exact optimum was not observed. The compiled circuit had depth 680 and 938 CZ gates.

The simulation and hardware experiments differ in problem size, circuit, shot count, and execution conditions. Their rates should not be treated as a controlled hardware-versus-simulation benchmark.

1. Problem formulation and experimental design

Portfolio selection balances expected return against risk. For a portfolio of N stocks, a binary vector x represents the selection decision: $x_i = 1$ means stock i is included and $x_i = 0$ means it is excluded. The return vector μ contains expected returns, and the covariance matrix Σ describes individual return variance and co-movement between pairs of stocks.

A representative mean-variance objective used in the project is:

$$C(x) = q \cdot x^T \Sigma x - \mu^T x$$

The first term penalizes portfolio risk; the second rewards expected return. The parameter q controls the risk-return trade-off. Because the objective is minimized, a lower cost is preferred. A negative cost is not itself a financial loss; it simply reflects the scale and sign convention of the mathematical objective.

The data are synthetic and designed for repeatable experiments. This removes dependence on market-data collection and supports clear comparisons, but means the results are not evidence of investment performance on real historical or live financial data.

Classical reference and quantum workflow

1 · Formulate	2 · Establish reference	3 · Run quantum method	4 · Validate
Encode stock decisions and constraints as binary variables and an objective.	Use exhaustive search for small feasible spaces to identify the reference minimum.	Simulate QAOA or encode features for a quantum-kernel model; submit selected circuits to IBM hardware.	Check cost, constraints, sample counts, ranking/classification metrics, and hardware limitations.

QAOA in this project

QAOA is a hybrid quantum-classical optimization method. A cost Hamiltonian encodes the objective and constraint penalties into a quantum circuit. A mixer circuit and cost circuit are arranged in p layers. The circuit is measured repeatedly to produce candidate bitstrings, while a classical optimizer adjusts the circuit parameters to improve expected objective value. A low p can keep circuits simpler, but does not guarantee convergence or high-quality sampling.

QAOA depth p is the number of QAOA layers; it is not the same as the compiled hardware circuit depth, which counts sequential circuit-operation layers after transpilation. Likewise, logical decision qubits are not necessarily the same as the circuit width reported after mapping and routing.

2. Level 1 — Beginner: six-stock QAOA

Research question

Can a QAOA circuit sample the optimal solution of a small portfolio problem when the exact classical answer is known? This level serves as a controlled learning case for objective encoding, simulation, and hardware execution.

Problem size and classical baseline

The task selects exactly three stocks from six. The number of feasible portfolios is C(6, 3) = 20. A classical exhaustive search evaluates all 20 valid combinations and establishes the reference optimum: Stocks 2, 3, and 6, with cost −0.503015.

Quantum implementation

The portfolio is encoded with six binary decision variables, represented by six logical qubits. The QAOA cost Hamiltonian contained 22 Pauli terms, and the experiment used p = 1. The simulator generated 4,096 measurements. A separate IBM Quantum hardware run on backend ibm_kingston used 1,024 shots.

Results

Evaluation	Shots	Valid portfolios / rate	Exact optimum observed	Interpretation
Classical exhaustive search	20 candidates	20 feasible combinations assessed	Reference optimum identified	Stocks 2, 3, 6; cost −0.503015
QAOA simulator	4,096	2,783 / 67.94%	Best sampled portfolio matched reference (exact hit count not reported here)	Useful simulation baseline
IBM ibm_kingston	1,024	609 / 59.47%	29 hits / 2.83%	Best valid sampled portfolio matched reference

Interpretation

The IBM run produced valid portfolios in 59.47% of shots and observed the exact optimum 29 times. This shows the circuit could return the reference solution on hardware, but most shots did not return that exact portfolio. A shot is one circuit execution followed by measurement. The valid-sample rate and the optimum-hit rate measure different things: a sample may satisfy constraints without being globally optimal.

This instance is small enough for exact classical enumeration. The contribution is therefore not a speedup claim; it is an implementation and validation exercise showing how QAOA outcomes can be compared with a known baseline.

Recorded IBM job ID: `db7mlh0r1c5c738kspdg` (1,024 shots).

3. Level 2 — Intermediate: quantum-kernel machine learning

Research question

Rather than directly searching for the minimum-cost portfolio, this level asks whether a quantum-state representation can help classify selected candidate portfolios as good or poor and preserve the reference ranking. This is a different task from QAOA optimization.

Dataset construction

The notebook considers 12 stocks and portfolios that select six stocks. There are C(12, 6) = 924 candidate portfolios. Each portfolio is represented as a 12-element binary feature vector; the full feature matrix has shape 924 × 12. Candidate costs determine labels for selected examples: good portfolios come from the lower-cost group and poor portfolios from the higher-cost group. Middle-ranking candidates are filtered out to form a binary classification problem.

Quantum-kernel SVM pipeline

Encode	Compare	Classify	Evaluate
A 12-qubit quantum feature map with two repetitions maps each 12-value portfolio vector to a quantum state $\|\varphi(x)\rangle$.	A fidelity kernel evaluates pairwise state similarity: $K(x,z) = \|\langle\varphi(x)\|\varphi(z)\rangle\|^2$.	A classical SVM uses the kernel matrix to classify labelled examples as good or poor.	Compare with a classical RBF-kernel SVM, and measure classification and ranking quality against the classical reference.

Why SVM?

A Support Vector Machine is a supervised classifier that can learn a decision boundary from examples. Kernel SVMs work with a similarity matrix, making SVM a natural choice for evaluating a quantum fidelity kernel. For the baseline, the same type of classifier used a standard classical radial basis function (RBF) kernel.

Results

Metric	Quantum fidelity-kernel SVM	Classical RBF-SVM
Initial evaluated split accuracy	100%	100%
Mean Spearman rank correlation across five evaluations	0.9469	0.9563
Ranking of exact reference optimum	Mean rank 1.4; first or second in the five recorded tests	Ranked first in all five recorded tests

Interpretation and scope

Both classifiers achieved 100% accuracy on the initial evaluated split. Because the data are synthetic and the labels are filtered by cost extremes, this result should not be interpreted as strong evidence of real-world generalization. The classical model had slightly higher mean Spearman correlation, so the experiments do not establish a quantum ranking advantage.

The IBM experiment in this level was a separate two-qubit feature-map-style circuit with 512 shots. It demonstrates execution of a related quantum circuit on real hardware; it was not execution or training of the complete 12-qubit fidelity-kernel SVM pipeline on the quantum processor.

Recorded IBM job ID: `db7n3e0r1c5c738ktlbg` (512 shots).

4. Level 3 — Advanced: constrained 20-stock QAOA

Research question

How does the optimization behave when the portfolio grows to 20 stocks and includes sector constraints? This level raises the number of logical decision variables and adds feasibility requirements to the cost Hamiltonian.

Problem definition

The model selects exactly K = 10 stocks from 20. The stocks are grouped into five sectors, with four stocks in each sector, and at most two stocks may be selected from any sector. Because exactly ten stocks must be selected in total, those constraints imply exactly two stocks per sector for any feasible solution. The feasible search space therefore contains C(4, 2)^5 = 6^5 = 7,776 portfolios.

The synthetic experiment used expected returns drawn within the configured range [0.05, 0.25] and a positive-semidefinite covariance construction $\Sigma = A A^{T}$. The notebook uses a fixed random seed (2026) to make the data construction reproducible.

Objective and constraint penalties

The project objective combines a mean-variance cost with penalty terms for violating the total-cardinality and sector constraints:

$$C(x) = q \cdot x^{T}\Sigma x - \mu^{T}x + \lambda_{card}(\Sigma_i x_i - K)^2 + \lambda_{sector} \Sigma_s(n_s - 2)^2$$

Here n_s is the number of selected stocks in sector s. The configured advanced-level risk coefficient is q = 0.5, and both penalty weights are $\lambda_{card} = 5$ and $\lambda_{sector} = 5$. Penalties make constraint-violating bitstrings costly, but they do not guarantee that every measured bitstring is feasible.

Classical reference and Hamiltonian validation

An exact classical evaluation of the 7,776 feasible portfolios identifies Stocks 2, 4, 5, 7, 11, 12, 13, 16, 18, and 20 as the reference optimum. The cost is −1.111739, with reported return score 1.736840 and risk 1.250202. The encoded cost Hamiltonian used 20 logical decision qubits and 211 Pauli terms. A numerical validation check on 101 bitstrings reported a maximum absolute discrepancy of approximately 5.116×10^{-13} between the tested representations.

Simulation and hardware results

Measure	QAOA simulator	IBM Quantum hardware
Shots	4,096	256
Valid portfolios	422 / 10.30%	1 / 0.39%
Exact classical optimum observed	Once	0 times
Best valid portfolio cost	−1.111739	−0.794773
Absolute best-sample cost gap	0.000000	0.316966
Circuit information	p = 1; 2 trainable parameters	Transpiled depth 680; 938 CZ gates

Interpretation

In simulation, the exact optimum was sampled once in 4,096 shots, so the best-sample cost gap was zero. This does not mean QAOA reliably produced the optimum; it means one sampled bitstring matched the classical solution. The pilot parameter search stopped at its 20-evaluation limit, so convergence was not established.

On hardware, the exact optimum was not observed. One valid portfolio was returned, and its cost was −0.794773, giving an absolute gap of 0.316966 from the classical optimum. Plausible contributors include circuit depth, hardware gate/readout noise, the limited 256 shots, strict constraints, and parameter quality. The run does not isolate the effect of any one factor.

Recorded IBM job ID: `db7n9iuil99s73fogb5g` (256 shots; 20 logical decision qubits).

5. IBM Quantum hardware experiments

Hardware execution is a key part of the project, but each experiment has a different purpose. The portfolio QAOA jobs evaluate sampled decisions against classical references; the intermediate QML job is a separate small circuit demonstration; and the Bell-state job is a basic entanglement-oriented circuit exercise. These runs are not a controlled benchmark against one another.

Recorded hardware runs

Experiment	Purpose	Shots	Recorded observations	Job ID
Beginner QAOA portfolio	Sample 6-stock portfolio solutions	1,024	609 valid (59.47%); optimum hit 29 times	db7mlh0r1c5c738kspdg
Bell-state demonstration	Execute a small two-qubit entanglement circuit	256	Counts: 00 = 85; 01 = 30; 10 = 22; 11 = 119	db7hinaraffc73faipbg
Intermediate feature-map-style demo	Demonstrate small-circuit hardware execution	512	Counts: 00 = 47; 01 = 164; 10 = 76; 11 = 225	db7n3e0r1c5c738ktlbg
Advanced QAOA portfolio	Sample constrained 20-decision portfolio	256	1 valid (0.39%); optimum not observed	db7n9iuil99s73fogb5g

What can be concluded from the hardware runs?

The Beginner hardware run demonstrates that the six-stock QAOA circuit could produce the known optimal portfolio on IBM hardware, but the hit rate was 29 out of 1,024 shots. The Intermediate job verifies that a small feature-map-style circuit was executed on hardware, but its two-qubit counts cannot be used as evidence that the full QML pipeline ran on the processor. The Advanced job shows a sharp practical challenge: only one measurement satisfied the portfolio constraints, and the exact optimum did not appear.

A measured bitstring must be decoded and checked against the constraints before it is treated as a valid portfolio. The lowest cost among all measured bitstrings is not necessarily the lowest cost among valid portfolios. For that reason, this report distinguishes valid-sample rate, exact-optimum hits, and best valid cost.

Why simulation and hardware can differ

A simulator can model ideal operations or selected noise models and does not automatically reproduce every physical source of error on a processor. Real devices introduce gate and measurement errors, and deeper circuits can make noise accumulation more consequential. Results also depend on circuit compilation, parameter optimization, the number of shots, and how the cost and constraints are encoded. Our experiments indicate these are relevant factors to investigate, but do not isolate causal contributions.

The Beginner and Advanced valid rates should not be compared as if only the hardware changed: the problems, circuits, constraints, and shot counts differ. A controlled study would hold the problem and evaluation protocol constant while varying one factor at a time.

6. Findings, limitations, and next steps

What the experiments support

- A classical baseline is essential: on these small instances, exhaustive search provides the reference optimum and a check on the quantum samples.
- QAOA can be implemented in simulation and used to generate candidate portfolio bitstrings; hardware runs can be compared with a known reference when the problem is small enough.
- A quantum fidelity-kernel pipeline can be constructed for labelled synthetic portfolios and compared with a classical kernel-based SVM.
- Feasibility is a first-class metric: a low objective value is useful only if the measured portfolio satisfies stock-count and sector constraints.
- The advanced simulation sampled the exact optimum once, whereas the advanced hardware run did not. This is evidence about these specific runs, not a universal statement about quantum computers.

Limitations

- Synthetic data: the expected returns and covariance inputs are designed for reproducibility and do not represent historical market behaviour.
- Small problem sizes: the classical feasible search spaces (20 and 7,776 candidates) are tractable, so no quantum speedup or quantum advantage is established.
- QML labels: the good/poor classification task uses filtered labels based on objective scores; 100% accuracy on the initial split is not proof of generalization to real markets or unseen problem distributions.
- Intermediate hardware scope: only a separate two-qubit feature-map-style circuit was run on hardware; the complete 12-qubit kernel/SVM pipeline was not run there.
- Advanced optimization: the parameter search stopped at the pilot evaluation limit; convergence was not established.
- Finite-shot and hardware effects: the runs have different shot counts and circuits; no controlled experiment isolates noise, depth, parameters, or sampling as the sole cause of the observed valid rates.

Recommended follow-up work

- Expand and systematically tune the QAOA parameter search; report objective-value trajectories and convergence criteria.
- Investigate constraint-aware mixers or alternative encodings that can reduce the proportion of invalid measured bitstrings.
- Repeat hardware runs with controlled changes to circuit depth, shot count, and optimization parameters, while recording backend calibration context where possible.
- Compare against appropriate classical optimizers and stronger baselines on larger, more challenging instances before assessing potential advantage.
- Test QML on additional splits, broader synthetic distributions, and eventually carefully sourced real data; separate classification performance from ranking performance.
- If hardware resources permit, explore whether a smaller end-to-end quantum-kernel evaluation can be executed on hardware, explicitly distinguishing circuit execution from classical kernel construction and SVM training.

Overall conclusion

The project is an end-to-end experimental study of portfolio optimization and learning with Qiskit: it combines classical reference calculations, QAOA simulation, a quantum-kernel SVM comparison, and selected IBM Quantum hardware runs. The most defensible conclusion is that the team demonstrated implementations and learned how to validate them, but the experiments did not demonstrate quantum advantage.

7. Reproducibility and project materials

Code repository

https://github.com/rahulydv39/Qiskit-Fall-Fest-2026

Repository contents include the README, reference list, and three notebooks: Beginner_QAOA.ipynb, Intermediate_Quantum_ML.ipynb, and Advanced_QAOA.ipynb. The notebooks document synthetic data generation, cost construction, simulation, evaluation, and recorded IBM hardware outputs.

To reproduce the local/simulation workflow, open the corresponding notebook in Jupyter or Google Colab, follow the setup cell, and run cells from top to bottom. Re-running a hardware cell may submit a new job and may depend on account access and available IBM Quantum resources. The hardware job IDs below identify recorded executions; access to detailed job records depends on IBM Quantum account permissions.

Recorded hardware job identifiers

Experiment	Job ID	Shots
Beginner QAOA portfolio	db7mlh0r1c5c738kspdg	1,024
Beginner Bell-state demonstration	db7hinaraffc73faipbg	256
Intermediate two-qubit feature-map-style demo	db7n3e0r1c5c738ktlbg	512
Advanced 20-logical-qubit QAOA	db7n9iuil99s73fogb5g	256

Software and method references

The following resources provide background and API documentation; they are not the source of the project-specific numerical outputs.

[1] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028. https://arxiv.org/abs/1411.4028.

[2] IBM Quantum Documentation. Quantum approximate optimization algorithm. QAOA tutorial.

[3] Qiskit Machine Learning. Quantum Kernel Machine Learning tutorial. Quantum-kernel tutorial.

[4] scikit-learn. Support Vector Machines. SVM documentation.

[5] Markowitz, H. (1952). Portfolio Selection. The Journal of Finance, 7(1), 77–91. doi:10.1111/j.1540-6261.1952.tb01525.x.

[6] Qiskit Finance. Portfolio Optimization tutorial. Portfolio optimization tutorial.

[7] Qiskit Aer. Simulator framework.

[8] IBM Quantum Runtime. API documentation.

AI assistance disclosure

The project repository documents use of ChatGPT (OpenAI) for quantum-computing explanations, code troubleshooting, experiment-output interpretation, and drafting/refining documentation and presentation material. The team is responsible for reviewing the implementation and verifying the reported results.

Thank you

Superposition Squad · Rahul Kumar · Alden Antony · Krishna Aggarwal